

DIGITAL SYSTEM ASSIGNMENT-3

4-BIT UP-DOWN COUNTER

using Verilog on RealDigital Boolean Board

EE25BTECH11011 - B.Navya

EE25BTECH11226 - Phani Vishnu Sree

1 OBJECTIVE

The goal of this project is to design and implement a **4-bit Synchronous Up-Down Counter** on the **RealDigital Boolean Board**. The design utilizes a clock divider to slow down the onboard 100MHz clock to a human-readable speed, uses a switch for direction control, and displays the output on four LEDs.

2 SYSTEM SPECIFICATIONS

- **Target Device:** Xilinx Spartan-7 (XC7S25) on RealDigital Boolean Board.
- **Input Signals:**
 - `Sc1k`: 100MHz System Clock.
 - `reset`: Asynchronous reset (Pushbutton).
 - `mode`: Direction control (High = Up, Low = Down).
- **Output Signals:**
 - `sCount[3:0]`: 4-bit binary count displayed on LEDs.
 - `mode_led`: LED indicator for the current counting direction.

3 DESIGN METHODOLOGY

3.1 Clock Division

The Boolean Board's oscillator runs at 100MHz. To make the counting visible, a 26-bit register (`clk_div`) is used as a frequency divider. The 25th bit is used as the `slow_clk`:

$$f_{slow} = \frac{100 \times 10^6 \text{ Hz}}{2^{26}} \approx 1.49 \text{ Hz} \quad (1)$$

3.2 Up-Down Logic

The counter uses a ternary operator to decide the next state.

- If mode is 1, `next_count = count + 1`.
- If mode is 0, `next_count = count - 1`.

4 SYSTEM BLOCK DIAGRAM

The system consists of three main internal components:

1. **Clock Divider:** A 26-bit counter that takes the 100MHz `Sc1k` and generates a `slow_clk` (approx 1.5Hz).
2. **Control Logic:** Uses the mode switch to select between an adder (+1) or subtractor (-1).
3. **Counter Register:** A 4-bit D-Flip-Flop based register that updates the state on the rising edge of `slow_clk`.

5 STATE MACHINE (FSM)

The counter acts as a Simple State Machine where each state represents a binary value from 0000 to 1111.

- **States:** 16 total states (S_0 to S_{15}).
- **Inputs:** mode (Up/Down) and `reset`.
- **Transitions:**
 - If mode=1: $S_n \rightarrow S_{n+1}$ (Wrap around $S_{15} \rightarrow S_0$).
 - If mode=0: $S_n \rightarrow S_{n-1}$ (Wrap around $S_0 \rightarrow S_{15}$).
 - If `reset`=1: Immediate transition to S_0 .

6 HARDWARE CONSTRAINTS (XDC)

The following pin assignments were used to map the internal signals to the Boolean Board's hardware components.

Signal	FPGA Pin	IO Standard	Hardware Component
Sclk	F14	LVC MOS33	100MHz Crystal
reset	H2	LVC MOS33	Pushbutton
mode	V2	LVC MOS33	Slide Switch
sCount[3]	A4	LVC MOS33	LED LD3
sCount[2]	B4	LVC MOS33	LED LD2
sCount[1]	A3	LVC MOS33	LED LD1
sCount[0]	B3	LVC MOS33	LED LD0
mode_led	G1	LVC MOS33	Status LED

XDC Pin Mapping for RealDigital Boolean Board

The .xdc file maps the internal Verilog ports to the physical pins of the FPGA.

```
1 // Clock
2 set_property PACKAGE_PIN F14 [get_ports Sclk]
3 set_property IOSTANDARD LVCMOS33 [get_ports Sclk]
4 create_clock -period 10.00 [get_ports Sclk]
5
6 // Reset button
7 set_property PACKAGE_PIN H2 [get_ports reset]
8 set_property IOSTANDARD LVCMOS33 [get_ports reset]
9
10 // Mode switch
11 set_property PACKAGE_PIN V2 [get_ports mode]
12 set_property IOSTANDARD LVCMOS33 [get_ports mode]
13
14 // Counter LEDs
15 set_property PACKAGE_PIN A4 [get_ports {sCount[3]}]
16 set_property PACKAGE_PIN B4 [get_ports {sCount[2]}]
17 set_property PACKAGE_PIN A3 [get_ports {sCount[1]}]
18 set_property PACKAGE_PIN B3 [get_ports {sCount[0]}]
19
20 set_property IOSTANDARD LVCMOS33 [get_ports {sCount[3]}]
21 set_property IOSTANDARD LVCMOS33 [get_ports {sCount[2]}]
22 set_property IOSTANDARD LVCMOS33 [get_ports {sCount[1]}]
23 set_property IOSTANDARD LVCMOS33 [get_ports {sCount[0]}]
24
25 // Mode LED (FIXED â new pin)
26 set_property PACKAGE_PIN G1 [get_ports mode_led]
27 set_property IOSTANDARD LVCMOS33 [get_ports mode_led]
```

boolean.xdc Constraints

7 VERILOG IMPLEMENTATION

```
1 module updown_count(  
2     input Sclk,  
3     input reset,  
4     input mode,  
5     output [3:0] sCount,  
6     output mode_led  
7 );  
8  
9     reg [3:0] count;  
10    reg [25:0] clk_div;  
11    wire slow_clk;  
12  
13    always @(posedge Sclk or posedge reset) begin  
14        if (reset)  
15            clk_div <= 0;  
16        else  
17            clk_div <= clk_div + 1;  
18    end  
19    // HARDWARE SETTING: assign slow_clk = clk_div[25];  
20    // SIMULATION SETTING: assign slow_clk = Sclk;  
21    assign slow_clk = clk_div[25];  
22  
23    wire [3:0] next_count;  
24    assign next_count = (mode) ? (count + 1) : (count - 1);  
25    assign sCount = count;  
26    assign mode_led = mode;  
27  
28    always @(posedge slow_clk or posedge reset) begin  
29        if (reset)  
30            count <= 4'b0000;  
31        else  
32            count <= next_count;  
33    end  
34 endmodule
```

Main Module: updown_count.v

8 SIMULATION AND VERIFICATION

A testbench was developed to verify the asynchronous reset and the transition between Up and Down counting modes.

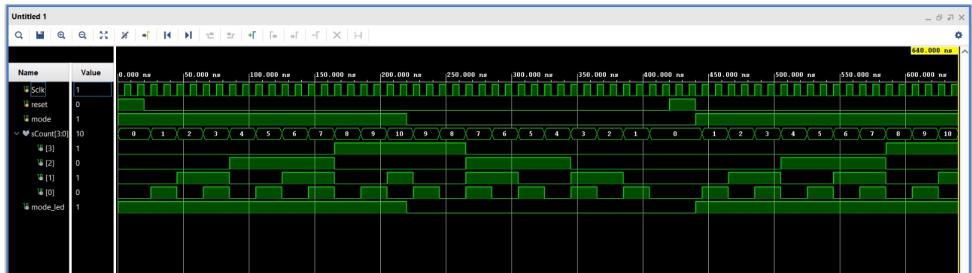
```
1 `timescale 1ns / 1ps  
2  
3 module updown_count_tb;  
4  
5     // Inputs  
6     reg Sclk;  
7     reg reset;  
8     reg mode;  
9  
10    // Outputs
```

```

11 wire [3:0] sCount;
12 wire mode_led;
13
14 // Instantiate the DUT (Device Under Test)
15 updown_count uut (
16     .Sclk(Sclk),
17     .reset(reset),
18     .mode(mode),
19     .sCount(sCount),
20     .mode_led(mode_led)
21 );
22 // Clock generation (10ns period â  same as your constraint)
23 initial begin
24     Sclk = 0;
25     forever #5 Sclk = ~Sclk; // 10ns clock
26 end
27 // Stimulus
28 initial begin // Initialize
29     reset = 1;
30     mode = 1; // Start with UP counting
31     #20;
32     reset = 0; // Release reset
33     // Run in UP mode
34     #200;
35     // Switch to DOWN mode
36     mode = 0;
37     #200;
38     // Apply reset again
39     reset = 1;
40     #20;
41     reset = 0;
42     // Again UP mode
43     mode = 1;
44     #200;
45     $stop;
46 end
47 endmodule

```

Testbench: updown_count_tb.v



8.1 CLOCK DIVISION FOR SIMULATION

During simulation, using `clk_div[25]` would require millions of nanoseconds of simulation time to see a single count change. To verify the logic efficiently, the clock divider was bypassed by assigning `slow_clk = Sclk` during the verification phase. For hardware deployment, the divider was restored to ensure visibility on the LEDs.

9 IMPLEMENTATION STEPS IN VIVADO

1. **Create Project:** Initialize a new RTL project selecting the Spartan-7 (xc7s25) device.
2. **Add Design Source:** Create `updown_count.v` and implement the clock division and counter logic.
3. **Create Constraints:** Add a new XDC file and map the ports to the RealDigital Boolean Board pins (F14, H2, V2, etc.).
4. **Run Synthesis:** Verify the RTL logic and check for syntax errors.
5. **Run Implementation:** Translate the design into FPGA logic gates and check timing.
6. **Generate Bitstream:** Create the `.bit` file for hardware programming.
7. **Hardware Manager:** Connect the Boolean Board via USB and program the device.

10 HARDWARE DEMONSTRATION

The working demonstration of the FPGA implementation can be viewed here:

Demo Video: <https://www.youtube.com/watch?v=W20vps-kHNY>

11 CONCLUSION

The 4-bit Up-Down counter was successfully implemented. The logic was verified through simulation and hardware testing, confirming correct operation of the clock divider, direction switch, and reset pushbutton.